Trail extension

Tree tracking — bug audit

97 distinct bugs found by 11 parallel forensic audits (Opus 4.7). Grouped by user-visible impact, with plain-language scenarios and severity tags.

At a glance

15	33	29	20
P0 · Critical Data loss, privacy, silent corruption	P1 · High Wrong tree structure, incorrect behavior	P2 · Medium Behavior drift, UX decay	P3 · Low Latent, cosmetic, perf

The 5 root causes behind everything

1. The tree's parent pointer is one global field

`tree.current` is a single mutable field, written from 15+ call sites (sidepanel clicks, tab activations, window focus, every navigation). Every new node is attached to whatever last wrote to it — not to the tab or link that caused the navigation.

2. `openerTabId` is referenced zero times in the entire codebase

Chrome tells us which tab opened which — for free. We throw it away. This is the single highest-impact, lowest-effort fix.

3. Mutations happen without a lock

The `treeMutex` exists but only protects bulk persistence. Every handler awaits 2–4 times before mutating and Chrome freely interleaves events. Combined with (1), every concurrent navigation can swap parents.

4. URL is the only identity

Duplicate tabs, middle-clicks, and multi-window use all collide into one node because `findByUrl` merges them.

5. MV3 service-worker lifecycle is ignored

Debounced 1.5s saves die with the worker. No `onSuspend` flush. Migration flags set before the async work completes.

The bugs, grouped by impact

Each card shows what the user sees, what's happening in the code, severity, and where to look.

Category 1: The tree shape doesn't match how you actually browse

The extension decides which node becomes the parent of the new one using a single global pointer (the sidepanel's 'current' selection). It never looks at which tab opened which. So the tree shape is shaped by your sidepanel attention, not by your browsing flow.

#1	P0 · Critical	**Every new tab attaches under your sidepanel's current selection — not under the tab that opened it**
	What you see	You're reading article A in Tab 1. You glance at Tab 2 (unrelated research) to copy a URL, then return to Tab 1 and Ctrl-click a link. The new tab appears as a child of the research you glanced at, not of article A.
	Why	`tree.navigate(data)` always attaches to `this.current`, which is mutated by 15+ call sites. `tab.openerTabId` — which Chrome provides for free — is referenced zero times in the entire codebase.
	Where	`nav-tree.ts:168; navigation-handler.ts:220`
#2	P0 · Critical	**Background-tab navigations silently reparent under the sidepanel**
	What you see	You're looking at node X in the sidepanel. A background tab in a different window refreshes or redirects. That redirect becomes a child of X.
	Why	Global `tree.current` is shared across all tabs and windows; no per-tab last-node tracking.
	Where	`nav-tree.ts:12, 58`
#3	P1 · High	**Multiple windows fight over one 'current' pointer**
	What you see	Two Chrome windows open. Window 2 (background) has an auto-refresh tab. That refresh stomps window 1's current. Your next nav in window 1 becomes a child of window 2's page.
	Why	`NavigationTree.current` is a single mutable field, written on every activation and navigation across all windows.
	Where	`nav-tree.ts:12; lifecycle.ts:139`
#4	P1 · High	**Session restore: parents decided by a parallel race**
	What you see	Chrome restores 10 tabs at startup. Tab 7 does a JS redirect after 500ms. At that moment `tree.current` is whatever tab happened to activate last — often unrelated. Tab 7's redirect becomes a child of that tab.
	Why	`onTabActivated` fires in parallel during restore; every firing writes `tree.current`.
	Where	`sync-open-tabs.ts:60; lifecycle.ts:130`

#5	P1 · High	**Ctrl+click to open the same URL in a new tab: no new branch is created**
	What you see	You want to explore a docs page from two angles — you middle-click it twice. Expected: two branches. Got: both tabs share one node.
	Why	`findByUrl` returns the first DFS hit and the new tab is registered to it.
	Where	`navigation-handler.ts:197; lifecycle.ts:37`
#6	P1 · High	**'Duplicate tab' loses the duplication signal**
	What you see	Right-click a tab → Duplicate. You expected a separate branch. No new node is created; the tree shows a single visit.
	Why	Duplicate tab has `openerTabId` set — but it's ignored. URL match routes to the same node.
	Where	`navigation-handler.ts:197`
#7	P2 · Medium	**Pinned tabs accumulate unrelated children over time**
	What you see	You pin Gmail. You click 50 different marketing links from Gmail over a week. Gmail's node ends up with 50 unrelated children because it was always `tree.current` when you clicked.
	Why	No special-case for pinned tabs in parent inference.
	Where	`navigation-handler.ts:98`
#8	P1 · High	**Redirect chains leave dead nodes everywhere**
	What you see	An OAuth login does 3 redirects (login.microsoft.com → aad.xyz → app). You end up on the real app, but the tree now has 3 useless intermediate nodes.
	Why	`redirectDetector.recordTabUrl` is called, but `redirectDetector.isRedirect` has zero call sites. The collected signal is thrown away.
	Where	`navigation-handler.ts:105; redirect-detection.ts:21`
#9	P1 · High	**Server vs client redirects, typed URLs, form submits — all treated identically**
	What you see	Chrome classifies every navigation via `transitionType` + `transitionQualifiers`. The extension ignores all of it.
	Why	Handler parameter is literally named `_transitionType` (underscore = intentionally unused).
	Where	`navigation-handler.ts:32`
#10	P1 · High	**SPA navigation (Gmail/Linear/GitHub) creates duplicate siblings**
	What you see	A single SPA click fires up to 3 webNavigation events (committed + historyStateUpdated + referenceFragmentUpdated) — all handled the same way. Depending on timing, you get a duplicate sibling node.
	Why	All three listeners share one handler, no deduplication via `documentId`.
	Where	`webnavigation.ts:14`

Category 2: Silent data loss

State that should persist — your pins, layouts, titles, undo history, recent navigations — sometimes disappears without a trace.

#11	P0 · Critical	**Deleting one node can silently delete hundreds of nested branches**
	What you see	You delete node N, expecting to remove N (and maybe its direct subtree). Instead, N's 500 descendants are lifted up to N's parent; any URLs that now share a key with siblings quietly merge, cascading recursively. Pin state, layout positions, child ordering — all lost with no confirmation.
	Why	`TreeNode.remove()` reparents children to grandparent; `mergeDuplicateChildren` then collapses URL duplicates recursively.
	Where	`tree-node.ts:80; nav-tree.ts:194`

#12	P0 · Critical	**OAuth popup can overwrite your tree's root node**
	What you see	A popup opens (window.open) on about:blank — tab maps to root. The popup then navigates to a real URL. The extension's 'if the old URL was untrackable, rewrite it in place' branch kicks in and overwrites your root's URL/title/favicon with whatever that popup visited.
	Why	The existing-node-with-untrackable-URL branch rewrites `node.data` in place without caring that the node is the tree's root.
	Where	`navigation-handler.ts:136–153`

#13	P0 · Critical	**Service worker dies within the 1.5s save window — up to 1.5s of navigations vanish**
	What you see	You open 5 tabs in quick succession; Chrome evicts the MV3 service worker before the debounced save timer fires. The timer dies with the worker. Nothing was persisted.
	Why	`cacheTree()` arms `setTimeout(1500)` but nothing calls `flushSaveNow` from `chrome.runtime.onSuspend`.
	Where	`tree-storage.ts:262; service-worker.ts has no onSuspend handler`

#14	P0 · Critical	**Cross-device sync loses edits when clocks disagree**
	What you see	Device B's clock is 10 min behind device A. B writes first but carries an older `savedAt` timestamp. On pull, A's conflict check uses `savedAt` as a vector clock — B's newer write is ignored, then on next push from A it silently clobbers B's changes.
	Why	Sync compares `savedAt` (client-generated) instead of server `updated_at`.
	Where	`sync-engine.ts:412`

#15	P0 · Critical	**NodeId collision after service worker restart**
	What you see	Service worker sleeps. Counter resets to 0. During async hydration, a navigation can create a new node with ID 6 that collides with a node about to be deserialized. `nodeMap.set(6, ...)` silently overwrites one. One node becomes unreachable via `findNodeById`.
	Why	`nodeIdCounter` is module-global and race-prone across async boundaries.
	Where	`tree-node.ts:2–22`

#16	P1 · High	**Renames and in-place URL rewrites break search silently**
	What you see	You rename a node. Searching by the new name returns nothing. Searching by the old name returns it.
	Why	Multiple paths do `node.data = {...}` without calling `refreshSearchEntry`. The MiniSearch index goes stale.
	Where	`navigation-handler.ts:137; editor-handlers.ts:341`

#17	P0 · Critical	**Your pinned-state and hand-arranged layout are lost during auto-merge**
	What you see	You've arranged positions and pinned node A. A duplicate collides (e.g. from session restore). The merge picks `base.layout ?? incoming.layout` and `Boolean(base.pinned || incoming.pinned)` — if `base` has no layout, incoming's is kept, but otherwise layout may vanish; pins stick permanently (you can't unpin once merged).
	Why	`mergeNodeData` uses boolean-OR for pin and left-wins for layout, no timestamp-based LWW.
	Where	`nav-tree.ts:72, 78`

#18	P1 · High	**Entering editor mode immediately erases the undo history it just created**
	What you see	Enter editor mode, make several edits, try to Undo back past the first edit — nothing happens.
	Why	`enterEditorMode` calls `snapshot(...)` then `CommandManager.clearHistory()` on the next line, wiping the snapshot.
	Where	`editor-handlers.ts:153–156`

#19	P1 · High	**Undo/redo strands tabs — your tabs lose their node mappings**
	What you see	You hit Undo on a branch you just created. Tabs that were mapped to nodes in that branch are now mapped to nothing. Next activation creates a brand-new node — your history duplicates.
	Why	`applyTree` calls `syncOpenTabs()` which does `tabManager.reset()` and then re-registers only tabs whose URL matches in the restored tree. Also, `syncOpenTabs` runs before `cacheTree(tree)`, so it sees the OLD tree.
	Where	`undo-handlers.ts:7–13`

#20	P1 · High	**'Tree doesn't follow my click' after temporary selection**
	What you see	You temporarily hover/preview a node, then click a tab. The tree doesn't move.
	Why	The temp-selection fallback queues `suppressNextActivate` for every tab in a recency list, but only clears the one it managed to focus. The rest stay suppressed until session end.
	Where	`tab-manager.ts:139–145; temporary-selection.ts:96–215`

#21	P1 · High	**Title stuck as URL forever**
	What you see	You load a page fast, navigate away before the title arrives. The node stores the URL as its title permanently — because the title-sync handler gates on URL-key match, which now fails.
	Why	`tab-sync.ts:30` requires `createUrlKey(node.data.url) === createUrlKey(tabUrl)` to fix the title.
	Where	`navigation-handler.ts:115; tab-sync.ts:30`

#22	P1 · High	**Orphan tab: after deleting a node, the mapped tab keeps no node**
	What you see	You delete a node N. A tab was mapped to N. The tab keeps browsing; every navigation in it silently does nothing because `tabManager.nodeByTab(tab)` still returns the dead id and `findNodeById(deadId)` returns null.
	Why	`deleteNode` in nav-tree.ts:194 never notifies `tabManager`.
	Where	`navigation-handler.ts:130–191`

Category 3: Concurrency — two handlers collide

Every handler (navigation, tab activated, tab removed, tab updated) mutates shared tree state with no lock, awaits multiple times, and Chrome freely interleaves events. Result: the tree's shape depends on microsecond-level timing.

#23	P0 · Critical	**Concurrent navigations in different tabs: parents get swapped**
	What you see	Two tabs commit a navigation in the same event-loop tick. Both handlers read `tree.current`, both call `setCurrentById` then `navigate()`. Between those two lines, the other handler slips in. The new node is attached to the wrong parent.
	Why	`setCurrentById + navigate` is a non-atomic compound. `treeMutex` exists but only guards persistence, not mutation.
	Where	`navigation-handler.ts:156–158, 218–230`

#24	P0 · Critical	**Service-worker boot race: registrations wiped mid-flight**
	What you see	SW wakes. `syncOpenTabs()` calls `tabManager.reset()` then `await browser.tabs.query({})`. Between those two, a webNav event fires, a handler registers a tab. `syncOpenTabs` then overwrites or erases that registration.
	Why	`reset()` also clears `ignoredTabs` and `suppressActivate` — unrelated in-flight intent is lost.
	Where	`sync-open-tabs.ts:17–47`

#25	P1 · High	**`syncOpenTabs` creates a dangling node on every service-worker wake**
	What you see	If your active tab's URL has session params (OAuth callbacks, shopping-cart IDs, Gmail `#inbox/...`), it doesn't match `findByUrl` on wake. `syncOpenTabs` calls `tree.navigate(data)` without setting current first — the new node attaches to the persisted current from last session, wherever that was. Tree slowly bloats with mis-parented nodes.
	Why	`sync-open-tabs.ts:80` does not set current before navigate.
	Where	`sync-open-tabs.ts:67–88`

#26	P1 · High	**Tab closed mid-handler — handler re-registers a dead tab id**
	What you see	Handler calls `await browser.tabs.get(tabId)`; the user closes the tab; `onTabRemoved` unregisters. Handler resumes, happily calls `tabManager.register(nodeId, deadTabId)`. Phantom tab entry persists until SW restart.
	Why	Multiple awaits between tab read and tree mutation.
	Where	`navigation-handler.ts:57, 214, 222`

#27	P1 · High	**Suppressed tab-activation still mutates the tree**
	What you see	`shouldSuppressActivate` is checked AFTER the tree is already navigated and the tab is already registered. Suppression doesn't prevent the mutation it was supposed to hide.
	Why	Lines 93–110 mutate before line 121 checks suppression.
	Where	`lifecycle.ts:86–123`

#28	P2 · Medium	**Sidepanel sees tree snapshots out of order**
	What you see	`notifyTreeUpdate` is awaited mid-handler. A concurrent handler can interleave, mutating the tree while the previous notify is still serializing. Sidepanel receives snapshots in a nondeterministic order.
	Why	No snapshot-then-broadcast separation; the broadcast reads live state.
	Where	`navigation-handler.ts:82, 246; lifecycle.ts:44, 65, 133`

#29	P1 · High	**Concurrent temp-selection `begin()` calls leave stranded suppressions**
	What you see	Two sessions start nearly together. Both see `activeSessionId === null` across the `await getActiveTab()`. Both write, second overwrites first; first's suppressions are never cleared.
	Why	`begin()` is not atomic.
	Where	`temporary-selection.ts:56–77`

Category 4: Special URLs and privacy

Incognito, blob:, file://, and about:blank URLs, plus URLs with credentials — the extension's handling of edge-case URLs leaks private data, misroutes real pages, and creates ghost nodes.

#30	P0 · Critical	**Incognito URLs leak into the public tree**
	What you see	Setting `privacy.skipIncognito` is honored ONLY in the navigation handler. The lifecycle handlers (onCreated, onActivated, onUpdated, focus-changed) all ignore `tab.incognito`. Your incognito URL becomes the active node and gets synced to Supabase.
	Why	Lifecycle handlers never check `tab.incognito` or `settingsManager.get().privacy.skipIncognito`.
	Where	`lifecycle.ts:18–149`

#31	P1 · High	**Pages with 'new tab' in their title hijack the root**
	What you see	Open a Stack Overflow thread titled 'How do I open a new tab...'. The extension detects 'new tab' in the title and routes it to your tree's root node.
	Why	`isNewTabContext` substring-matches the title.
	Where	`url-utils.ts:38–42`

#32	P1 · High	**Custom new-tab-page extensions are tracked as regular pages**
	What you see	Flip side of #31. A custom NTP with a title like 'Start page' doesn't match 'new tab' and gets tracked as if it were a normal site.
	Why	Same substring logic.
	Where	`url-utils.ts:38–42`

#33	P2 · Medium	**blob: / file:// / chrome-untrusted:// URLs silently skipped, stale mappings persist**
	What you see	You open a PDF from a blob: URL. The tab has no node. But the tab's PRIOR mapping (from an earlier http page) persists. Next meta-patch for that tab updates the WRONG URL's title.
	Why	`shouldTrackUrl` filters but doesn't un-register stale mappings.
	Where	`url-utils.ts:45–52; navigation-handler.ts:93`

#34	P2 · Medium	**SPA hash-routing sites are invisible in the tree**
	What you see	Older Angular / docsites using `site.com/#/intro` → `/#/install`. The handler strips the hash during URL-key computation, so every route change is a no-op. Your entire in-app navigation doesn't appear in the tree.
	Why	`createUrlKey` sets `url.hash = ""`.
	Where	`url-key.ts:50`

#35	P3 · Low	**URLs with auth credentials may be stored**
	What you see	`https://user:pass@site.com` — if Chrome doesn't strip before handoff, credentials land in your tree and sync payload.
	Why	No credential-stripping in the URL normalization.
	Where	`url-utils.ts`

Category 5: Storage, quota, performance

Writes aren't atomic, migrations race, stub-archives re-download every time, and startup stalls grow linearly with tree size.

#36	P1 · High	**Synced archives re-download from Supabase every time you open them**
	What you see	Another device wrote an archive. You open it once: extension downloads, caches blob locally. You open it again: it downloads again. And again.
	Why	`fetchRemoteTreePayload` saves the blob but never calls `unstubArchiveEntry`. That function has zero call sites.
	Where	`sync-engine.ts:444–479; tree-storage.ts:716–745`

#37	P1 · High	**First-boot migration race can expose legacy-shape data**
	What you see	`ensureMigrated` sets `migrated = true` BEFORE the migration read. A concurrent caller sees 'done', reads legacy-shape data, caches it, then real migration overwrites. The window is brief but real on slow first boots after upgrade.
	Why	Flag set before await instead of after.
	Where	`tree-storage.ts:900–936`

#38	P2 · Medium	**Archive deletion can leak blob data (quota growth)**
	What you see	Extension writes index (removing archive X), then removes the blob. Crash between = index says 'gone', blob stays. Over time: quota fills.
	Why	Multi-key writes aren't crash-atomic; no startup GC.
	Where	`tree-storage.ts:703–710`

#39	P2 · Medium	**Startup stalls for large trees**
	What you see	Power users with 10k+ nodes: every service-worker wake blocks on MiniSearch index rebuild. MV3 wakes are frequent — every tab event, every RPC.
	Why	`constructFromRoot` does `searchIndex.addAll(docs)` on every hydration; index isn't persisted.
	Where	`nav-tree.ts:27–44`

#40	P2 · Medium	**Close-oldest picks your first-opened 'anchor' tab on a fresh profile**
	What you see	Fresh Chrome profile: every tab has `lastAccessed=0`. When you hit the tab limit, close-oldest sorts by `lastUsed - 0` ties, picks whichever got into the data structure first. That's usually the tab you care most about.
	Why	`getLastUsed` uses `|| 0` collapsing 'untracked' with epoch.
	Where	`tab-lru.ts:43; tab-manager.ts:207`

Category 6: Behavior drift

Smaller issues that accumulate: stale counts, spurious focus jumps, unnecessary writes.

#41 P2 · Medium — Visit count inflates 2× on active tabs

What you see	`markNodeVisited` is called twice on the same navigation in the active-tab path.
Why	Redundant call sites at lines 146 and 188–189.
Where	`navigation-handler.ts:146, 163, 186–190`

#42 P2 · Medium — `setCurrentById` silently no-ops on missing ids

What you see	The sidepanel says 'set current to X'. If X was merged away by a background cleanup, nothing happens — silently. The sidepanel thinks it succeeded.
Why	Method returns void, 10+ call sites never check.
Where	`nav-tree.ts:321–325`

#43 P2 · Medium — Alt-tab amplifies write pressure

What you see	Every window focus change fires `tabManager.touch + markNodeVisited + setCurrentById + cacheTree + notifyTreeUpdate` — even when the active tab hasn't changed.
Why	No short-circuit for unchanged active tab.
Where	`lifecycle.ts:139–149`

#44 P2 · Medium — Focus jumps unexpectedly after deleting a node during background load

What you see	`prevCurrentId` is snapshotted early. If a concurrent delete merges that node away, `setCurrentById(prevCurrentId)` no-ops — current stays on the newly-created background node, violating the 'only-move-on-active-tab' invariant.
Why	Stale ID used after merge.
Where	`navigation-handler.ts:127, 148–167, 226–227`

#45 P2 · Medium — Pin state doesn't sync when you pin a tab in Chrome

What you see	You manually pin a tab in Chrome. The tree's node stays unpinned because the same-URL navigation branch uses `metaPatch = {title, favicon}` only.
Why	`pinned` never applied in the same-URL branch.
Where	`navigation-handler.ts:113–184`

#46 P3 · Low — Canonical URL display stays stale

What you see	You navigate to `https://example.com/Path/`, then later `Example.com/Path`. Same URL-key, so same node. But display URL stays whatever the first visit was.
Why	`metaPatch` omits URL, even when case/slash differs.
Where	`navigation-handler.ts:120`

#47	P3 · Low	**CSS bug: `getChildren` allocates frozen copy per call**
	What you see	Tree renders on a 5k-node tree → 20k allocations per render.
	Why	`Object.freeze([...this.children])` on every call, in hot paths.
	Where	`tree-node.ts:112–114`

#48	P2 · Medium	**`tabsByNode` returns an ephemeral empty Set for missing keys**
	What you see	Consumer might think they're mutating the live set; they mutate a throwaway.
	Why	API smell: `?? new Set()` conflates missing with empty.
	Where	`tab-index.ts:45–47`

Category 7: Latent and edge cases

Issues that are real but only bite in rare configurations, or that set up future regressions.

#49	P0 · Critical	**Drag-drop reparent with ancestor-as-child creates a cycle → infinite loop**
	What you see	If a reparent RPC is ever called with an ancestor as the child, `attachChild` accepts it. DFS iteration then infinite-loops. SW hangs until Chrome kills it.
	Why	No ancestry check in `TreeNode.attachChild` beyond `child === this`.
	Where	`tree-node.ts:42–63`

#50	P2 · Medium	**`lru.remove()` leaves `latestNodeId` stale**
	What you see	When a replacement is found, only `latestTabId` is updated. A consumer reading both gets inconsistent pairs.
	Why	Known invariant violation with an apologetic code comment.
	Where	`tab-lru.ts:46–64`

#51	P2 · Medium	**`suppressNextTouch` is single-token; rapid-fire overwrites**
	What you see	Two near-simultaneous suppression requests: the second overwrites, the first's touch goes through.
	Why	Single field vs the Map used for `suppressActivate`.
	Where	`tab-manager.ts:15, 115`

#52	P3 · Low	**`nodeCount` bandaged with `Math.max(0, …)` — hides double-decrements**
	What you see	`tree.size()` under-reports until next reconstruction; UI badges show wrong counts.
	Why	Clamping hides bugs instead of asserting invariants.
	Where	`nav-tree.ts:205, 267`

#53	P3 · Low	**No `onDetached` / `onAttached` listeners — dragging tabs between windows is latent**
	What you see	Works today only because `windowId` isn't stored. Any future window-scoped feature silently drifts.
	Why	Grep: zero matches for these listeners.
	Where	`lifecycle.ts:168–174`

#54	P3 · Low	**Dead file: `navigation/history-api.ts` is 0 bytes**
	What you see	Empty module left over from earlier work. Confusing to readers; potential import issue.
	Why	File is empty.
	Where	`navigation/history-api.ts`

#55	P3 · Low	**`clearArchivesOnly` / `clearAllTrees` use one `try` wrapping the whole loop**
	What you see	One failed `removeItem` aborts the rest; partial state is left.
	Why	try{} around loop = all-or-nothing, but cleanup is supposed to be best-effort.
	Where	`tree-storage.ts:787–820`

#56	P3 · Low	**`dirty=false` set before the save's `await` resolves**
	What you see	If SW dies mid-save, `dirty` is already lowered. `catch` resets it on error, but not on termination.
	Why	Flag order problem.
	Where	`tree-storage.ts:437–439`

#57	P2 · Medium	**Temp-selection `preview`/`cancel`/`accept` unregister tabs on focus failure**
	What you see	`browser.tabs.update` throws on transient failure (window closing, OS denies focus). Code interprets as 'tab gone' and unregisters — but the tab still exists and its user mapping was just destroyed.
	Why	Speculative cleanup masquerading as real cleanup.
	Where	`temporary-selection.ts:102–124, 172–173, 222–227`

#58	P2 · Medium	**`archiveActiveTree → switchTree` doesn't flush in-flight saves first**	
	What you see	Pending `cacheTree` debounce sees a different tree than the archive write. Also, `tabManager.reset()` runs before `setActiveTree` commits — if that throws, tabs are orphaned across all nodes.	
	Why	No `flushSaveNow()` before archive/switch.	
	Where	`tree-storage.ts:660; tree-handlers.ts:240`	
#59	P3 · Low	**Editor auto-exit not idempotent under rapid activation**	
	What you see	Two concurrent `onTabActivated` both see `editor active`, both await save. Depending on impl, either double-saves or throws.	
	Why	No 'already saving' guard.	
	Where	`lifecycle.ts:74–81`	
#60	P3 · Low	**`mergeNodeData` left-wins on layout — hand-arranged positions can vanish**	
	What you see	Merge picks `base.layout ?? incoming.layout`. If `base` has no layout but `incoming` does, fine. Otherwise incoming's hand-arranged positions are discarded.	
	Why	No timestamp-based LWW.	
	Where	`nav-tree.ts:72`	
#61	P3 · Low	**Pinned-once stays pinned forever**	
	What you see	`pinned = Boolean(base.pinned || incoming.pinned)`. Any history of pinning wins during merges; you can't unpin.	
	Why	Boolean-OR instead of LWW.	
	Where	`nav-tree.ts:78`	
#62	P2 · Medium	**Back-forward cache restores may be invisible**	
	What you see	Browser back button from BFCache may not fire webNav. Your history doesn't reflect going back.	
	Why	Platform-dependent, browser-level.	
	Where	`platform`	
#63	P3 · Low	**`createUrlKey` catch-branch produces keys in a different namespace than the try-branch**	
	What you see	Malformed URLs caught by `try { new URL(...) } catch {}` get keys without `https://` prefix. Parallel duplicates can exist.	
	Why	Different normalization on success vs failure path.	
	Where	`url-key.ts:44–67`	

#64	P3 · Low	**No `onErrorOccurred` listener — aborted navigations can still create nodes**
	What you see	URL aborts mid-flight. Later `pushState` on a 'retry' button creates a node as if the initial nav succeeded.
	Why	Webnav lifecycle half-wired.
	Where	`webnavigation.ts`

#65	P1 · High	**`onTabRemoved` queries wrong window**
	What you see	Close a tab in a background window. `currentWindow:true` returns last-focused window's active tab. Your tree's current yanks to the unrelated foreground tab.
	Why	`currentWindow:true` is wrong scope.
	Where	`lifecycle.ts:47–67`

#66	P2 · Medium	**`onTabRemoved` notifies before caching**
	What you see	Sidepanel sees pre-mutation state; concurrent `cacheTree` can land the old view into persistence.
	Why	Order inverted vs other handlers.
	Where	`lifecycle.ts:65–66`

#67	P1 · High	**`onTabUpdated` ignores `changeInfo.url`**
	What you see	Same-document URL changes delivered by `tabs.update` (not webNav) never flow into tree state. Title/favicon drift.
	Why	Handler only checks audio/title/favicon.
	Where	`lifecycle.ts:151–166`

#68	P2 · Medium	**`onTabActivated` sets current without `shouldTrackUrl` check**
	What you see	Tab with stale mapping to a chrome:// URL still triggers focus jump to the old node.
	Why	Gate only exists in the `nodeId === undefined` branch.
	Where	`lifecycle.ts:83–136`

#69	P2 · Medium	**`ignoredTabs` never garbage-collected for tabs that never register**
	What you see	Slow leak. More worryingly: when Chrome eventually reuses TabIds, the reused id is silently ignored.
	Why	No cleanup path.
	Where	`tab-manager.ts:17, 147`

#70	P1 · High	**`register()` accepts any nodeId without tree validation**
	What you see	If a caller passes a deleted node id, `TabIndex` happily creates a ghost node entry. `openNodes()` returns phantoms.
	Why	TabManager doesn't know about the tree.
	Where	`tab-manager.ts:43–58`

#71	P1 · High	**`touch` ignores its `_nodeId` param, trusts the index instead**
	What you see	The param is prefixed `_` to advertise 'unused'. Caller intent diverges from reality; no warning logged.
	Why	Parameter is a lie.
	Where	`tab-manager.ts:34–39`

#72	P3 · Low	**`touch` overwrites intentionally-old `lastUsed` with `Date.now()`**
	What you see	`syncOpenTabs` sets accurate timestamps from `tab.lastAccessed`. Any later `touch()` without a `lastUsed` arg clobbers to now. All tabs eventually cluster.
	Why	Overwrite semantics.
	Where	`tab-manager.ts:34; tab-lru.ts:12`

#73	P2 · Medium	**`findByUrl` picks arbitrary DFS winner on URL duplicates**
	What you see	Two subtrees share a URL. `findByUrl` always returns the leftmost-DFS hit. One subtree becomes a black hole for all re-visits to that URL.
	Why	No proximity heuristic.
	Where	`nav-tree.ts:311–319`

#74	P3 · Low	**`findByUrl` is O(n) per new tab; 10k-node tree stalls**
	What you see	Linear scan runs on every first-nav of every new tab. Perceptible at scale.
	Why	No URL-key → node map.
	Where	`nav-tree.ts:311; navigation-handler.ts:197`

#75	P2 · Medium	**`closeOldest` throws but Chrome actually closed the tab**
	What you see	`onTabRemoved` fires second unregister; `closeOldestUntil` over-closes. You asked to keep N tabs; lose N+1.
	Why	`try { tabs.remove(); unregister() } catch { return null }` — catch doesn't clean up.
	Where	`tab-manager.ts:237–243`

#76	P2 · Medium	**Editor snapshot taken mid-mutation window**
	What you see	Between `loadTree(treeId)` and `structuredClone(toJSON(tree))`, nav events can mutate the live tree. Rollback captures a partially-mutated state.
	Why	No lock around snapshot.
	Where	`editor-handlers.ts:135–151`

#77	P1 · High	**`saveEditorMode` coalescing drops intermediate work**
	What you see	Concurrent callers share one in-flight promise. Second navigation proceeds thinking editor exited, but active tree is about to be swapped — node lands in wrong tree.
	Why	Coalescing doesn't preserve ordering vs navigation side effects.
	Where	`editor-handlers.ts:221–237`

Category 8: Settings, blocklist, recency

Settings propagation has gaps — blocklist effectively off during the first 50-200ms of every service-worker wake, settings changes from other contexts don't reach the background, and visit counts are quietly dropped by a flawed timestamp guard.

#78	P0 · Critical	**Blocklist is off for 50–200ms on every service-worker wake — privacy leak**
	What you see	You blocklisted `*.bank.com`. Chrome evicts the service worker (every few minutes of inactivity). You then open a banking tab. During the brief window before `settingsManager.init()` resolves and the blocklist is compiled, navigation handlers fire with `shouldTrackUrl` returning true — the banking URL gets tracked.
	Why	`cachedBlocklist = () => false` as the default (track everything). `settingsManager.init()` is fire-and-forget; `loadSettings → import → setUrlBlocklist` is a chained async cascade.
	Where	`url-utils.ts:57; settings.ts:218–233; service-worker.ts:42–49`

#79	P1 · High	**Visit count drops silently when clocks mix**
	What you see	Callers pass two clocks interchangeably: `Date.now()` vs `tab.lastAccessed`. These aren't synchronized sub-ms; `lastAccessed` can trail. Writing `Date.now()` then `tab.lastAccessed` no-ops because the guard is `<= prevAt`. `visitCount` is undercounted; `lastVisitedAt` freezes at the wrong value.
	Why	`markNodeVisited` uses `if (nextAtRaw <= prevAtRaw) return false;` — strict-equal collapses rapid calls too.
	Where	`node-recency.ts:19`

#80	P1 · High	**Tree-switch permanently deletes restore-URLs on partial restore**
	What you see	You switch trees with 30 saved-open-tabs. Restore opens 12, then the service worker dies or `browser.tabs.create` fails. The other 18 URLs are deleted from storage forever — no recovery path.
	Why	`getOpenTabs` deletes the saved map entry before it returns. Loop in tree-handlers.ts:264 catches+continues on throws; SW termination also loses the URLs.
	Where	`tab-restoration.ts:19–26; tree-handlers.ts:251, 264–281`

#81	P2 · Medium	**Settings/flags changes from other contexts don't propagate until SW restart**
	What you see	Settings UI writes through RPC — works. But any direct write from onboarding, sidepanel, or popup (via `settingsItem.setValue`) doesn't update the in-memory state in the background.
	Why	No `storage.onChanged` / `.watch()` listener in the background. In-memory state only refreshed on SW boot.
	Where	`settings.ts; flags.ts`

#82	P2 · Medium	**Future default-settings fields will overwrite user overrides**
	What you see	`saveSettings` spreads `{...defaultSettings, ...next}`. Any field added to `defaultSettings` in a later version will stomp a user's saved override when they patch any other field — because `next` was loaded under the old defaults.
	Why	Merge semantics: new defaults applied AFTER old saved values in the spread.
	Where	`settings.ts:203`

#83	P3 · Low	**Migration fails silently on out-of-order entries**
	What you see	If a contributor appends `{from:2,to:3}` before `{from:1,to:2}` in the `MIGRATIONS` array, `{2→3}` is skipped during that session.
	Why	Single-pass `for` loop filtered by `m.from !== current`. No `while (current < CURRENT_VERSION)` guard.
	Where	`migrations.ts:54–69`

Category 9: Sidepanel and UI sync

The sidepanel receives tree updates via a custom pub-sub with debouncing. Several reasons are missing from the immediate-broadcast list, transient state leaks across unrelated updates, and bootstrap can overwrite live state with a stale snapshot.

#84	P0 · Critical	**Delete / Backspace fires while Rename or confirm dialogs are open — silent deletion**
	What you see	You open the Rename dialog. You tab out of the input onto the Cancel button, then hit Backspace. The shortcut handler thinks you're not in an overlay (it doesn't check `renameTarget` or `deleteTreeConfirm.open`) and deletes the selected node.
	Why	`overlayOpen` check is incomplete — doesn't cover rename, delete-confirm, command palette, sync-conflict dialogs.
	Where	`useShortcuts.ts:55–56, 293–308`
#85	P1 · High	**Rename doesn't show up in the sidepanel for up to 200ms**
	What you see	You rename a node. Dialog closes. Tree display still shows the old title briefly (longer if another debounced update coalesces).
	Why	`rename` is not in `IMMEDIATE_REASONS` — falls into the 200ms debounce bucket.
	Where	`notifications.ts:44–57`
#86	P1 · High	**Click-to-navigate feels sluggish — 200ms debounce**
	What you see	Clicking a link and the sidepanel's current-indicator update feels laggy. `navigate` is the default reason; also debounced. But `set-current` (sidepanel-driven) is immediate — inconsistent perceived latency.
	Why	`IMMEDIATE_REASONS` misses `navigate`, `tabs`, `temp-selection`, `sync`.
	Where	`notifications.ts:215`
#87	P1 · High	**Stale `viewIntent` leaks across unrelated tree updates**
	What you see	You rename a node (which emits a focus-to-node intent). Then a background tab fires a `tabs`-reason update. The stale `viewIntent` from the rename is still in store state — the graph re-focuses/zooms to the renamed node.
	Why	`applyTreeState` resolves `nextViewIntent = payload.view ?? state.graph.viewIntent`. `clearViewIntent` has zero callers.
	Where	`store.ts:310–315, 632–635`
#88	P1 · High	**Bootstrap races the first broadcast and overwrites live state**
	What you see	`getTreeState()` RPC resolves with snapshot T0. Before the React effect commits, a navigation fires `TREE_UPDATED` with snapshot T1 — which is applied. Then `bootstrapGraph(T0)` runs and clobbers T1 with stale data.
	Why	No timestamp guard or 'already bootstrapped' flag.
	Where	`useBootstrap.ts:63–78; useRuntimeMessages.ts:16–24`

#89	P2 · Medium	**Retry re-seeds store, wipes live tab state**
	What you see	User hits 'retry' after a transient glitch. `bootstrapGraph` replaces tree/pins/openNodes but leaves `collapsedNodeIds`/`search`/`selectedNodeIds` alone. Mismatch with `applyTreeState`'s tree-switch semantics → inconsistent state.
	Why	Two different reducers for 'apply snapshot' with different reset rules.
	Where	`useBootstrap.ts:97–101; store.ts:283–302`

#90	P2 · Medium	**Every tree update allocates new Sets, Maps, PinnedManagers → cascading re-renders**
	What you see	`applyTreeState` returns fresh `Set` / `Map` / `PinnedManager` instances on every payload, even when contents haven't changed. `TrailGraph` and `PinnedDropdown` re-render on every tab-count ping.
	Why	No structural-equality diff before allocation.
	Where	`store.ts:305–383; PinnedDropdown.tsx:46`

#91	P2 · Medium	**Debounced `view` intent discarded on coalesce**
	What you see	Two debounced calls arrive. The second omits `view`. The first's `view` (with `focusNodeId`) is dropped because `pendingArgs = { ..., ...(view ? { view } : {}) }` replaces wholesale.
	Why	No merge semantics for pendingArgs.
	Where	`notifications.ts:229–239`

#92	P3 · Low	**`Shift+Delete` double-fires recursive delete**
	What you see	With tinykeys, `Shift+Delete` matches BOTH the `"Delete"` binding (with `shiftKey` branching) AND the separate `"Shift+Delete"` binding. Both handlers call `deleteNodes(..., true)` → RPC fires twice.
	Why	Redundant bindings.
	Where	`useShortcuts.ts:293–308`

Category 10: Additional distinct bugs

Bugs that overlap thematically with earlier categories but have unique mechanics worth calling out separately.

#93	P0 · Critical	**Activating a pre-existing untracked tab creates a false child of your current node**
	What you see	You install the extension or the service worker wakes up. You have 20 tabs already open, none tracked yet. You click tab #7 — extension creates a brand-new node for it as a CHILD of whatever you're currently viewing in the sidepanel. All 20 pre-existing tabs get false parent edges based on the order you happen to click them.
	Why	`onTabActivated`'s fallback branch calls `tree.navigate(data)` which uses `this.current` as the parent. Should attach as a root or skip until a real navigation happens.
	Where	`lifecycle.ts:101–112`

#94	P1 · High	**Merge leaves `tree.current` as a zombie pointer**
	What you see	After `mergeDuplicateChildren` collapses a subtree, if `tree.current` was a DESCENDANT of the merged-away subtree (not a direct child), the code doesn't catch it. `tree.current` stays pointing at a node that's still in `nodeMap` but unreachable via DFS from root — invisible to serialization but `getCurrentId()` still returns its id. On save and reload, current silently resets to root.
	Why	Cursor-patch at nav-tree.ts:108 only checks `this.current.id === child.id`, not ancestor-chain intersection.
	Where	`nav-tree.ts:84–117`

#95	P3 · Low	**`syncOpenTabs` refreshes LRU but not `node.lastVisited`**
	What you see	Service worker restarts. `tabManager` recency is refreshed from `tab.lastAccessed`. But the tree's own `node.lastVisited` is stale — UI ordering by 'recent' drifts after every boot until nodes get re-visited.
	Why	No `markNodeVisited` pass after `tabManager.touch` loop in syncOpenTabs.
	Where	`sync-open-tabs.ts:49–53`

#96	P3 · Low	**`WINDOW_ID_NONE` check is insufficient for transient hidden windows**
	What you see	Devtools pop-outs, picture-in-picture hosts, macOS workspace transitions fire `onFocusChanged` for hidden-type windows. Extension doesn't filter on `windows.get(id).type === 'normal'`.
	Why	Only `WINDOW_ID_NONE` is filtered.
	Where	`lifecycle.ts:140`

#97	P3 · Low	**`setUrlBlocklist` cache key is semantically fragile**
	What you see	Key is `patterns.join('\n')`. Reordering `['a.com', 'b.com']` → `['b.com', 'a.com']` is semantically identical but recompiles. `['a\nb']` (pathological) collides with `['a', 'b']`.
	Why	String-join hashing has no canonicalization.
	Where	`url-utils.ts:60–62`

The 5 highest-leverage fixes

Most of the 97 bugs collapse into these five architectural changes.

1. Use `tab.openerTabId` as the primary parent signal

Fall back to `tree.current` only when there's no opener. Kills bugs #1, #2, #4, #5, #6, #7, #23, #73.

Low effort · Huge impact

2. Wrap every mutating handler in `withTreeLock`

Export it from tree-storage.ts. Serializes all mutations behind the same mutex as persistence. Kills bugs #23, #24, #26, #28, #29, and the whole concurrency class.

Low effort · Huge impact

3. Consume `details.transitionType` and `documentId`

Chrome already classifies redirects and dedupes SPA events. Use the browser's own signals instead of our broken `redirectDetector`. Kills bugs #8, #9, #10, #12.

Medium effort · High impact

4. Stop using the module-global `nodeIdCounter` — switch to uuidv7

`uuidv7` is already imported in tree-storage.ts. Kills bug #15 and the whole ID-race class.

Medium effort · High impact

5. Encapsulate all mutations behind `NavigationTree` methods

Make `TreeNode.data` externally readonly. Expose `tree.updateNodeData(id, patch)` that refreshes the search index and emits change events. Kills bugs #16, #31, #44, #45, #42.

Medium effort · High impact

Compiled from 11 parallel Opus 4.7 forensic audits covering: race conditions, navigation handler, tab lifecycle, tree integrity, tab manager, SPA / redirects, storage, parent inference, sidepanel UI sync, undo / editor, and settings / recency / blocklist.